

标注数据

标注工具

- [LasbelStudio](#)
- [Labelme](#)

数据组织

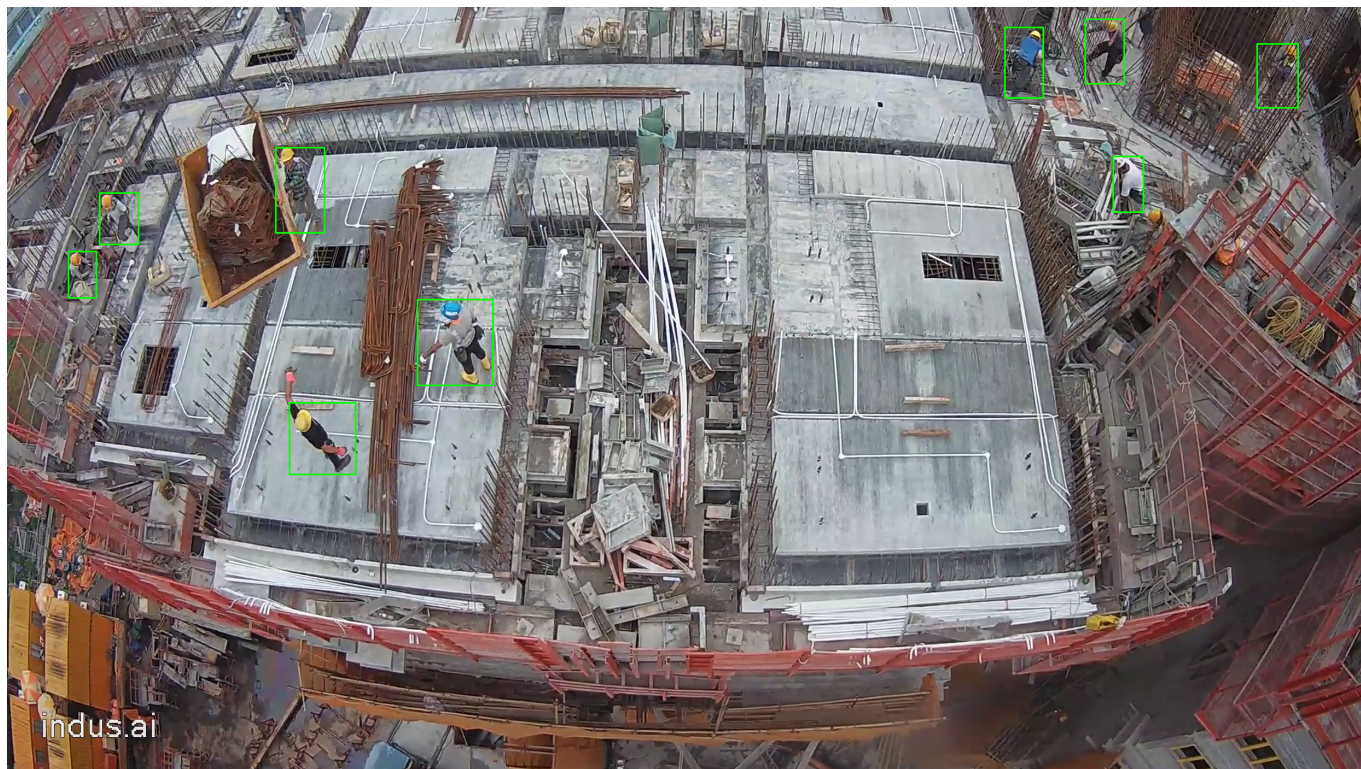
yolov5训练数据组织结构如下所示：

```
$ tree dataset/
dataset/
├── train
│   ├── images
│   │   ├── crane1.2019-05-21.10-00.10200.png
│   │   └── crane1.2019-05-21.10-00.10800.png
│   └── labels
│       ├── crane1.2019-05-21.10-00.10200.txt
│       └── crane1.2019-05-21.10-00.10800.txt
└── val
    ├── images
    │   ├── crane1.2019-05-23.08-00.10500.png
    │   └── crane1.2019-05-23.08-00.10800.png
    └── labels
        ├── crane1.2019-05-23.08-00.10500.txt
        └── crane1.2019-05-23.08-00.10800.txt

$ cat dataset/train/labels/crane1.2019-05-21.10-00.10800.txt
0 0.2317 0.5643 0.0486 0.0936
0 0.9329 0.0900 0.0301 0.0836
...
0 0.0554 0.3500 0.0205 0.0614
```

每个txt文件对应同名的图片 每一行代表一个检测框，总共五列,分别是类别，中心点x，中心点y，宽，高，这些值都是相对于图片的宽高的比例，如下所示：

类别	中心点x	中心点y	宽	高
0	0.2317	0.5643	0.0486	0.0936
0	0.9329	0.0900	0.0301	0.0836
...	...	...	...	...
0	0.0554	0.3500	0.0205	0.0614



训练环境搭建

操作系统：Ubuntu 20.04 64位

安装python虚拟环境（miniconda）

Miniconda下载地址：https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
sh Miniconda3-latest-Linux-x86_64.sh -b
#将conda路径加入Path
~/miniconda3/bin/conda init bash
source ~/.bashrc
#创建虚拟环境
conda create -n yolov5 python=3.8 -y
#激活虚拟环境
conda activate yolov5
```

```
#创建工作目录
mkdir yolov5_train
cd yolov5_train
#下载yolov5代码
git clone https://github.com/ultralytics/yolov5
cd yolov5
#安装依赖
```

```
pip install -r requirements.txt
cd ..
```

训练

训练命令如下所示：

```
export PYTHONPATH="$PWD/yolov5:$PYTHONPATH"
img_size=640
weights="yolov5s.pt"
mkdir -p "runs/train"

python yolov5/train.py \
    --data dataset.yaml \
    --img $img_size \
    --weights $weights \
    --batch-size 16 \
    --device 0 \
    --epochs 200 \
    --exist-ok \
    --project "runs/train" \
    --name "worker.${img_size}.${weights}"
```

其中dataset.yaml文件内容如下所示：

```
$ cat dataset.yaml
path: "Absolute path to your dataset"
train: ["train"]
val: ["val"]

names: ['worker']
```

详细内容可参考教程：https://docs.ultralytics.com/yolov5/tutorials/train_custom_data/#local-logging

模型导出

onnx

onnx文件是一种通用的模型文件，可以在不同的平台上部署，导出成onnx文件可以通过onnxruntime部署到jetson上，提高推理速度。

onnx导出

训练完成后，会在runs/train文件夹下生成对应的模型文件，如下所示：

```
$ tree runs/train/worker.640.yolov5s.pt
runs/train/worker.640.yolov5s.pt
├── confusion_matrix.png
├── F1_curve.png
├── hyp.yaml
├── labels_correlogram.jpg
├── labels.jpg
├── opt.yaml
├── P_curve.png
├── PR_curve.png
├── R_curve.png
├── results.csv
├── results.png
├── train_batch0.jpg
├── train_batch1.jpg
├── train_batch2.jpg
├── val_batch0_labels.jpg
├── val_batch0_pred.jpg
├── val_batch1_labels.jpg
├── val_batch1_pred.jpg
├── val_batch2_labels.jpg
├── val_batch2_pred.jpg
└── weights
    ├── best.pt
    └── last.pt
```

模型导出命令如下所示：

```
cd yolov5
python export.py --weights ../runs/train/worker.640.yolov5s.pt/weights/
```

在原模型父目录下会生成best.onnx文件，如下所示：

```
$ tree ../runs/train/worker.640.yolov5s.pt/weights/
../runs/train/worker.640.yolov5s.pt/weights/
├── best.onnx
├── best.pt
└── last.pt
```